Multithreading in Python

What is Multithreading?

Multithreading is a technique where **multiple threads** run **concurrently** within a single process. It helps in performing **multiple tasks simultaneously**, especially useful in **I/O-bound** operations.

For example:

- Reading/writing files while handling user input.
- Downloading multiple files at the same time.

Key Concepts:

Term	Explanation
Thread	A lightweight unit of a process that can run in parallel.
Main Thread	The thread in which the Python program starts.
Child Thread	Threads created by the main thread for other tasks.

How to Use Multithreading in Python

Python provides a built-in `threading` module.

Example:

```
import threading
import time

def display():
    for i in range(5):
        print(f"Thread running: {i}")
        time.sleep(1)

# Creating thread
t = threading.Thread(target=display)

# Starting thread
t.start()

# Main thread continues
for i in range(5):
    print(f"Main thread: {i}")
    time.sleep(1
```

Important Methods

Method	Description
`start()`	Starts the thread.
`join()`	Waits for the thread to complete.
`is_alive()`	Checks if thread is still running.

Why Not for CPU-bound Tasks?

Due to **GIL (Global Interpreter Lock)** in Python, only one thread executes at a time in CPU-bound operations. So, for **CPU-bound** tasks, prefer **multiprocessing** instead of multithreading.

Use Cases:

- Web scraping multiple sites at once.
- File downloads.
- Concurrent I/O operations (API calls, file reads).

What is Multiprocessing?

Multiprocessing is a technique where **multiple processes** run in parallel, each with its own Python interpreter and memory space.
It is used to **bypass the Global Interpreter Lock (GIL)** and is ideal for **CPU-bound tasks** like:

- Image processing
- Heavy calculations
- Data analysis

Key Concepts

Term	Description
Process	An independent unit of execution with its own memory.
Main Process	The process in which Python script starts.
Child Process	New processes started by the main process using `multiprocessing` module.

Example:

```
from multiprocessing import Process

def calculate_square(numbers):
```

```
    print("Squares:")
    for n in numbers:
        print(f"{n}^2 = {n*n}")

def calculate_cube(numbers):
    print("Cubes:")
    for n in numbers:
        print(f"{n}^3 = {n*n*n}")

if __name__ == "__main__":
    nums = [2, 3, 4, 5]

    # Create processes
    p1 = Process(target=calculate_square, args=(nums,))
    p2 = Process(target=calculate_cube, args=(nums,))

    # Start processes
    p1.start()
    p2.start()

    # Wait for processes to finish
    p1.join()
    p2.join()

    print("Done with multiprocessing!")
```

Output (order may vary):

```
Squares:
2^2 = 4
3^2 = 9
4^2 = 16
5^2 = 25

Cubes:
2^3 = 8
3^3 = 27
4^3 = 64
5^3 = 125
Done with multiprocessing!
```

Important Methods

Method	Description
`start()`	Starts the process.
`join()`	Waits for the process to finish.
`is_alive()`	Checks if the process is still running.

Comparison: Multithreading vs Multiprocessing

Feature	Multithreading	Multiprocessing
Ideal for	I/O-bound tasks	CPU-bound tasks
Memory	Shared	Separate memory space
Speed	Slower for CPU tasks	Faster for CPU tasks
GIL Affected?	Yes	No

Additional Concepts

1. **Process Pooling** (for multiple tasks):

```
from multiprocessing import Pool

def square(n):
    return n * n

with Pool(processes=4) as pool:
    result = pool.map(square, [1, 2, 3, 4])
    print(result)  # Output: [1, 4, 9, 16]
```

2. **Sharing Data (Optional)**:
   Use `Value`, `Array`, or `Queue` from `multiprocessing` to share data between processes.

Summary

Feature	Details
Module	`multiprocessing`
Best for	CPU-bound tasks
Independent memory?	Yes
GIL limitation?	No
Common tools	`Process`, `Pool`, `Queue`, `Lock`

Notes:

- `Process(target=func, args=(...))` — allows passing functions and their arguments to separate processes.
- `start()` — begins execution of each process.
- `join()` — waits for process to complete.
- Output order is **not guaranteed** as processes run **independently** and **simultaneously**.